

Types in concurrency



Emilio Tuosto @ GSSI

Reliable Software by Construction

5-10 October
Lorentz Center, Leiden

– My assignment –

[...]

Tutorials are intended to be 90 minutes long. The goal is to create sufficient common knowledge that will enable interactive discussions later on.

We have in mind a broad exposition of a specific development [global types & choreographies] on the topic, ideally with pointers to the relevant literature.

From Jorge's email

[...]

Tutorials are intended to be 90 minutes long. The goal is to create sufficient common knowledge that will enable interactive discussions later on.

We have in mind a broad exposition of a specific development [global types & choreographies] on the topic, ideally with pointers to the relevant literature.

[...]

audience includes people from academia, but also practitioners from industry
who may not be familiar with the style/notation typical of formal methods presentations.

A non-exhaustive list

Acta Informatica (2005) 42(2/3): 79–81
DOI 10.1007/s00236-005-0174-2

EDITORIAL

Rocco De Nicola · Davide Sangiorgi

Types in concurrency

Published online: 11 October 2005
© Springer-Verlag 2005

Traditionally types have played a very limited role in concurrency; they were essentially used for specifying the nature of the exchanged values. The picture has changed after the introduction of formalisms for dealing with systems of mobile processes. In addition to the classical use of types in sequential languages, e.g. for static detection of run time error, enhancing program readability, memory management, abstracting from implementation details, types have emerged as an important tool for:

- specifying interfaces and possible interactions
- controlling process mobility
- controlling resource usage
- controlling access rights
- developing new proof techniques
- improving efficiency of verification algorithms
- guaranteeing integrity and confidentiality of data

"(4) to make programs easier to understand" [24, p. 233]

...and more recently, e.g.,

- ▶ foundations
 - ▶ connection with logics
 - ▶ expressivity
 - ▶ new models of coordination
- ▶ programming
- ▶ static analysis
- ▶ run-time verification (monitoring)
- ▶ information flow analysis
- ▶ subtyping
- ▶ ...

A taste of program comprehension

Erlang

```
1 ping(0, Pong_PID) ->
2   Pong_PID ! finished,
3   io:format("Ping finished~n", []);
4
5 ping(N, Pong_PID) ->
6   Pong_PID ! {ping, self()},
7   receive
8     pong ->
9       io:format("Ping received pong~n", []),
10      ping(N - 1, Pong_PID).
```

```
11 pong() ->
12   receive
13     finished ->
14       io:format("Pong finished~n", []);
15     {ping, Ping_PID} ->
16       io:format("Pong received ping~n", []),
17       Ping_PID ! pong,
18       pong()
19   end.
```

A taste of program comprehension

```
1 ping(0, Pong_PID) ->
2   Pong_PID ! finished,
3   io:format("Ping finished~n", []);

4 ping(N, Pong_PID) ->
5   Pong_PID ! {ping, self()},
6   receive
7     pong ->
8       io:format("Ping received pong~n", []),
9   end,
10  ping(N - 1, Pong_PID).
```

```
11 pong() ->
12  receive
13    finished ->
14      io:format("Pong finished~n", []);
15    {ping, Ping_PID} ->
16      io:format("Pong received ping~n", []),
17      Ping_PID ! pong,
18      pong()
19  end.
```

Erlang

- Embodies the actor model [17, 1] ... back to '73!

A taste of program comprehension

```
1 ping(0, Pong_PID) ->
2   Pong_PID ! finished,
3   io:format("Ping finished~n", []);
4
5 ping(N, Pong_PID) ->
6   Pong_PID ! {ping, self()},
7   receive
8     pong ->
9       io:format("Ping received pong~n", []),
10      end,
11      ping(N - 1, Pong_PID).
```

```
11 pong() ->
12   receive
13     finished ->
14       io:format("Pong finished~n", []);
15     {ping, Ping_PID} ->
16       io:format("Pong received ping~n", []),
17       Ping_PID ! pong,
18       pong()
19   end.
```

Erlang

- ▶ Embodies the **actor model** [17, 1] ... back to '73!
- ▶ Message passing + functional programming

A taste of program comprehension

```
1 ping(0, Pong_PID) ->
2   Pong_PID ! finished,
3   io:format("Ping finished~n", []);

4 ping(N, Pong_PID) ->
5   Pong_PID ! {ping, self()},
6   receive
7     pong ->
8       io:format("Ping received pong~n", []),
9   end,
10  ping(N - 1, Pong_PID).
```

```
11 pong() ->
12  receive
13    finished ->
14      io:format("Pong finished~n", []);
15    {ping, Ping_PID} ->
16      io:format("Pong received ping~n", []),
17      Ping_PID ! pong,
18      pong()
19  end.
```

Erlang

- ▶ Embodies the **actor model** [17, 1] ... back to '73!
- ▶ Message passing + functional programming
- ▶ Dynamic creation of threads

A taste of program comprehension

```
1 ping(0, Pong_PID) ->
2   Pong_PID ! finished,
3   io:format("Ping finished~n", []);
4
5 ping(N, Pong_PID) ->
6   Pong_PID ! {ping, self()},
7   receive
8     pong ->
9       io:format("Ping received pong~n", []),
10      end,
11   ping(N - 1, Pong_PID).
```

```
11 pong() ->
12   receive
13     finished ->
14       io:format("Pong finished~n", []);
15     {ping, Ping_PID} ->
16       io:format("Pong received ping~n", []),
17       Ping_PID ! pong,
18       pong()
19   end.
```

Erlang

- Embodies the **actor model** [17, 1] ... back to '73!
- Message passing + functional programming
- Dynamic creation of threads
- **Asynchrony by design!** (FIFO buffers)

*mailboxes in
Erlang's jargon*

Let's spell out the program

A taste of program comprehension

```
1 ping(0, Pong_PID) ->
2   Pong_PID ! finished,
3   io:format("Ping finished~n", []);
4
5 ping(N, Pong_PID) ->
6   Pong_PID ! {ping, self()},
7   receive
8     pong ->
9       io:format("Ping received pong~n", []),
10      end,
11      ping(N - 1, Pong_PID).
```

```
11 pong() ->
12   receive
13     finished ->
14       io:format("Pong finished~n", []);
15     {ping, Ping_PID} ->
16       io:format("Pong received ping~n", []),
17       Ping_PID ! pong,
18       pong()
19   end.
```

Erlang

- ▶ Embodies the **actor model** [17, 1] ... back to '73!
- ▶ Message passing + functional programming
- ▶ Dynamic creation of threads
- ▶ **Asynchrony by design!** (FIFO buffers)

*mailboxes in
Erlang's jargon*

Let's spell out the program

- ▶ it takes some effort to get to the point with e.g., students

A taste of program comprehension

```
1 ping(0, Pong_PID) ->
2   Pong_PID ! finished,
3   io:format("Ping finished~n", []);
4
5 ping(N, Pong_PID) ->
6   Pong_PID ! {ping, self()},
7   receive
8     pong ->
9       io:format("Ping received pong~n", []),
10      end,
11      ping(N - 1, Pong_PID).
```

```
11 pong() ->
12   receive
13     finished ->
14       io:format("Pong finished~n", []);
15     {ping, Ping_PID} ->
16       io:format("Pong received ping~n", []),
17       Ping_PID ! pong,
18       pong()
19   end.
```

Erlang

- ▶ Embodies the **actor model** [17, 1] ... back to '73!
- ▶ Message passing + functional programming
- ▶ Dynamic creation of threads
- ▶ **Asynchrony by design!** (FIFO buffers)

*mailboxes in
Erlang's jargon*

Let's spell out the program

- ▶ it takes some effort to get to the point with e.g., students

Let's get some help...

Friendlier representations with (formal) models

Local behaviour [18, 20, 4]



CFSMs: FIFO buffers as well

$$c = \mu t. (!\text{ping}. ?\text{pong}. t \oplus !\text{finished})$$

$$s = \mu t. (? \text{ping}. !\text{pong}. t \& ? \text{finished})$$

Friendlier representations with (formal) models

Local behaviour [18, 20, 4]

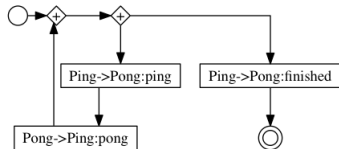


CFSMs: FIFO buffers as well

$$c = \mu t. (!\text{ping}. ?\text{pong}.t \oplus !\text{finished})$$

$$s = \mu t. (? \text{ping}. !\text{pong}.t \& ? \text{finished})$$

Global behaviour [20, 11, 23]

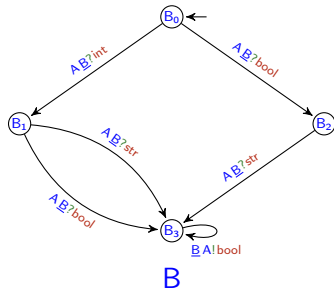
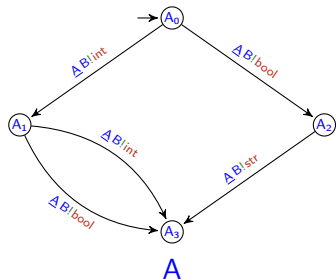


... "synchronous" distributed workflow

$$\mu t. C \rightarrow S : \begin{cases} \text{finished} \\ \text{ping}.S \rightarrow C : \text{pong}.t \end{cases}$$

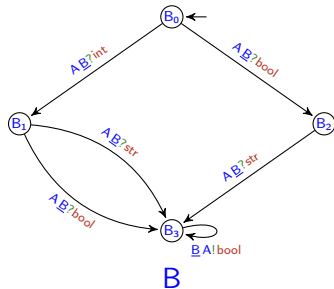
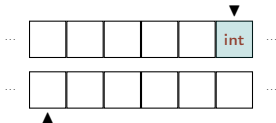
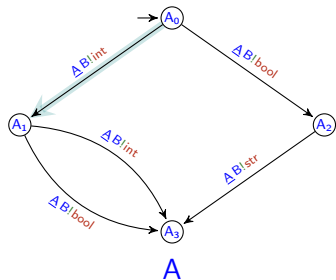
A formal model...informally

Communicating systems



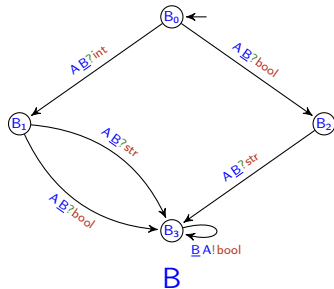
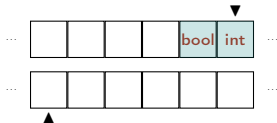
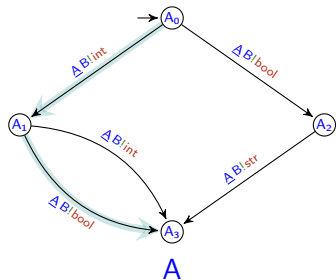
A formal model...informally

Communicating systems



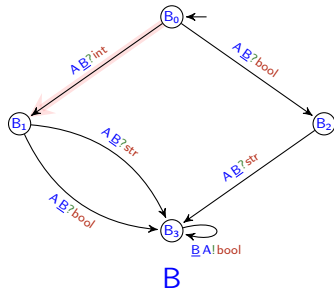
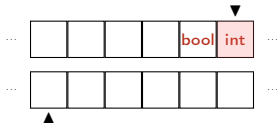
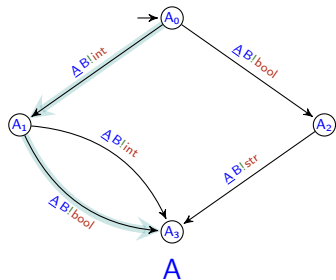
A formal model...informally

Communicating systems



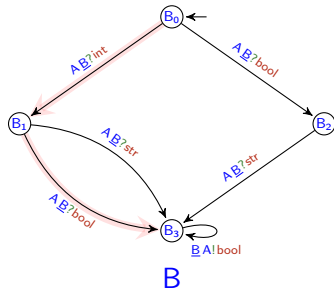
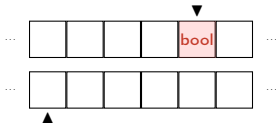
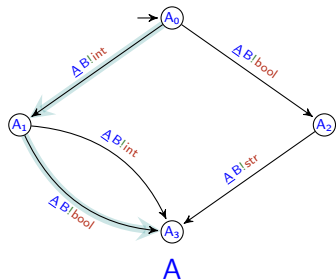
A formal model...informally

Communicating systems



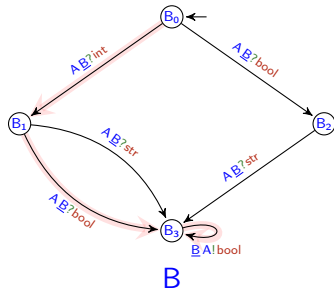
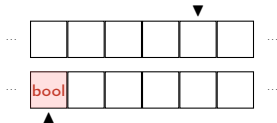
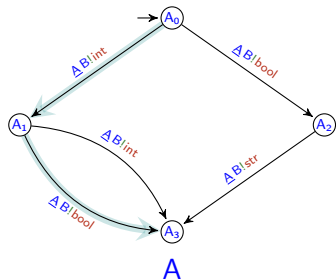
A formal model...informally

Communicating systems



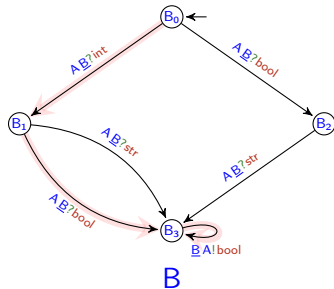
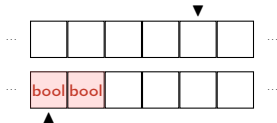
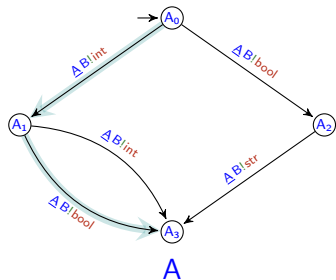
A formal model...informally

Communicating systems



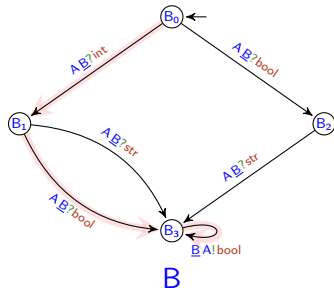
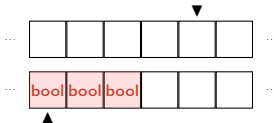
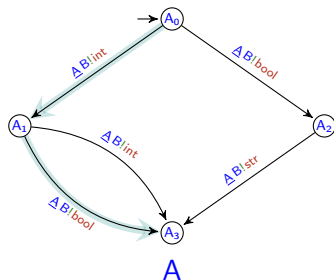
A formal model...informally

Communicating systems



A formal model...informally

Communicating systems



- ▶ a **communicating finite-state machine** (CFSM) is an FSA^a whose transitions are input/output actions executed by a single participant and whose states are all accepting
- ▶ a **communicating system** is a finite map^b assigning to a participant **A** a CFSM executing communications of **A**

^aAkin (UML) state machine

^bA.k.a. dictionary

On program correctness

So what?

On program correctness

So what?

- Now we have a model of the behaviour

So what?

- ▶ Now we have a model of the behaviour
- ▶ Let's use it to reason on correctness, assuming
 - ▶ no failures
 - ▶ reliable communications
 - ▶ abstract away from data and "local" computation

On program correctness

```
1 ping(0, Pong_PID) ->
2   Pong_PID ! finished,
3   io:format("Ping finished~n", []);
4
5 ping(N, Pong_PID) ->
6   Pong_PID ! {ping, self()},
7   receive
8     pong ->
9     io:format("Ping received pong~n", []),
10    end,
11    ping(N - 1, Pong_PID).
```

```
11 pong() ->
12   receive
13     finished ->
14     io:format("Pong finished~n", []);
15     {ping, Ping_PID} ->
16     io:format("Pong received ping~n", []),
17     Ping_PID ! pong,
18     pong()
19   end.
```

```
20 start() ->
21   Pong_PID = spawn(pingpong, pong, []),
22   spawn(pingpong, ping, [3, Pong_PID]).
```

So what?

- ▶ Now we have a model of the behaviour
- ▶ Let's use it to reason on correctness, assuming
 - ▶ no failures
 - ▶ reliable communications
 - ▶ abstract away from data and "local" computation

Q: Testing...our understanding

Does the test on lines 20-22 pass?

On program correctness

```
1 ping(0, Pong_PID) ->
2   Pong_PID ! finished,
3   io:format("Ping finished~n", []);
4
5 ping(N, Pong_PID) ->
6   Pong_PID ! {ping, self()},
7   receive
8     pong ->
9     io:format("Ping received pong~n", []),
10    end,
11    ping(N - 1, Pong_PID).
```

```
11 pong() ->
12   receive
13     finished ->
14     io:format("Pong finished~n", []);
15     {ping, Ping_PID} ->
16     io:format("Pong received ping~n", []),
17     Ping_PID ! pong,
18     pong()
19   end.
```

```
20 start() ->
21   Pong_PID = spawn(pingpong, pong, []),
22   spawn(pingpong, ping, [3, Pong_PID]).
```

So what?

- ▶ Now we have a model of the behaviour
- ▶ Let's use it to reason on correctness, assuming
 - ▶ no failures
 - ▶ reliable communications
 - ▶ abstract away from data and "local" computation

Q: Testing...our understanding

Does the test on lines 20-22 pass?
Is the program terminating?

Concurrency/distribution is challenging!

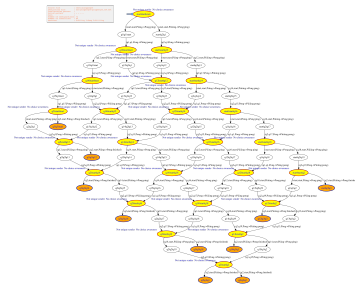
Another test:

```
1 start() ->
2   Pong_PID = spawn(pingpong, pong, []),
3   spawn(pingpong, ping1, [3, Pong_PID]),
4   spawn(pingpong, ping2, [5, Pong_PID]).
```

Concurrency/distribution is challenging!

Another test:

```
1 start() ->  
2   Pong_PID = spawn(pingpong, pong, []),  
3   spawn(pingpong, ping1, [3, Pong_PID]),  
4   spawn(pingpong, ping2, [5, Pong_PID]).
```



– Choreographies –

"Top-down"

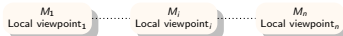
Quoting W3C [21]

"Using the Web Services Choreography specification, a **contract** containing a global definition of the common **ordering conditions and constraints** under which **messages** are exchanged, is produced that describes, from a **global viewpoint** [...] observable behaviour of all the parties involved. **Each party** can then use the global definition to **build and test solutions that conform to it**. The global specification is in turn **realised by combination of the resulting local systems** [...]"

Synchrony

Choreography G
global viewpoint

Asynchrony



"Top-down"

Quoting W3C [21]

"Using the Web Services Choreography specification, a **contract** containing a global definition of the common **ordering conditions and constraints** under which **messages** are exchanged, is produced that describes, from a **global viewpoint** [...] observable behaviour of all the parties involved. **Each party** can then use the global definition to **build and test solutions that conform to it**. The global specification is in turn **realised by combination of** the resulting **local systems** [...]"

Synchrony

Choreography G
global viewpoint

Asynchrony

M_1
Local viewpoint₁

M_i
Local viewpoint_i

M_n
Local viewpoint_n

spec, no code

"Top-down"

Quoting W3C [21]

"Using the Web Services Choreography specification, a **contract** containing a global definition of the common **ordering conditions and constraints** under which **messages** are exchanged, is produced that describes, from a **global viewpoint** [...] observable behaviour of all the parties involved. **Each party** can then use the global definition to **build and test solutions that conform to it**. The global specification is in turn **realised by combination of** the resulting **local systems** [...]"

Synchrony

Choreography G
global viewpoint

Well-formedness

Asynchrony

M_1
Local viewpoint₁

M_j
Local viewpoint_j

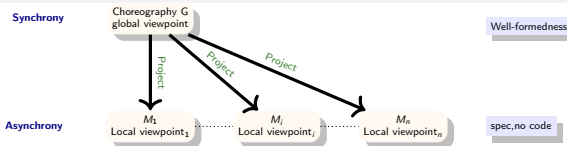
M_n
Local viewpoint_n

spec, no code

"Top-down"

Quoting W3C [21]

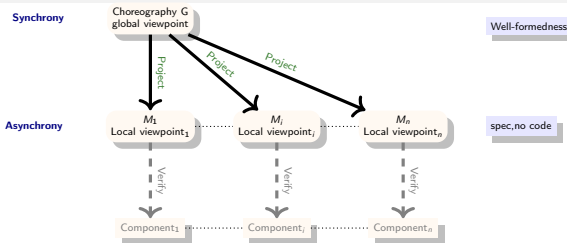
"Using the Web Services Choreography specification, a **contract** containing a global definition of the common **ordering conditions and constraints** under which **messages** are exchanged, is produced that describes, from a **global viewpoint** [...] observable behaviour of all the parties involved. **Each party** can then use the global definition to **build and test solutions that conform to it**. The global specification is in turn **realised by combination of the resulting local systems** [...]"



"Top-down"

Quoting W3C [21]

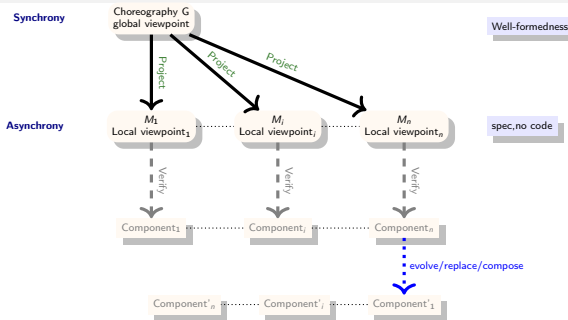
"Using the Web Services Choreography specification, a **contract** containing a global definition of the common **ordering conditions and constraints** under which **messages** are exchanged, is produced that describes, from a **global viewpoint** [...] observable behaviour of all the parties involved. **Each party** can then use the global definition to **build and test solutions that conform to it**. The global specification is in turn **realised by combination of the resulting local systems** [...]"



"Top-down" & "bottom-up" [23]

Quoting W3C [21]

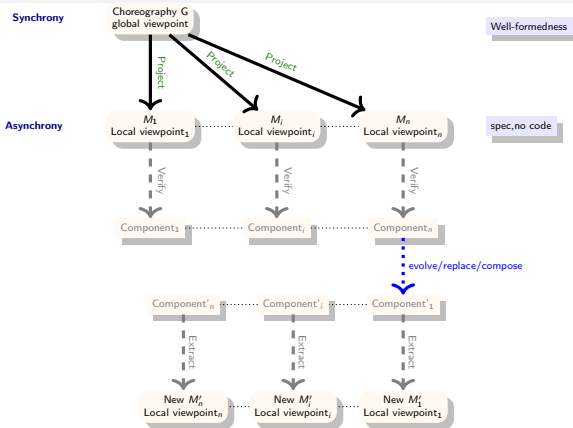
"Using the Web Services Choreography specification, a **contract** containing a global definition of the common **ordering conditions and constraints** under which **messages** are exchanged, is produced that describes, from a **global viewpoint** [...] observable behaviour of all the parties involved. **Each party** can then use the global definition to **build and test solutions that conform to it**. The global specification is in turn **realised by combination of the resulting local systems** [...]"



"Top-down" & "bottom-up" [23]

Quoting W3C [21]

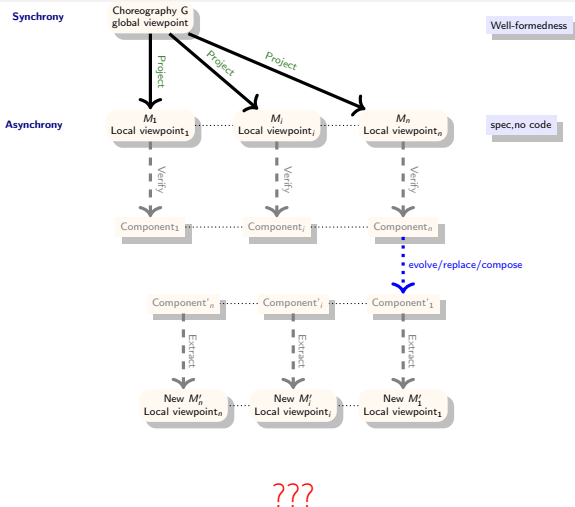
"Using the Web Services Choreography specification, a **contract** containing a global definition of the common **ordering conditions and constraints** under which **messages** are exchanged, is produced that describes, from a **global viewpoint** [...] observable behaviour of all the parties involved. **Each party** can then use the global definition to **build and test solutions that conform to it**. The global specification is in turn **realised by combination of the resulting local systems** [...]"



"Top-down" & "bottom-up" [23]

Quoting W3C [21]

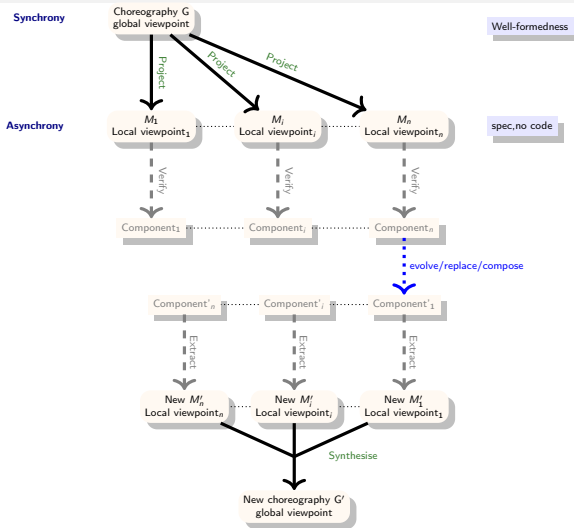
"Using the Web Services Choreography specification, a **contract** containing a global definition of the common **ordering conditions and constraints** under which **messages** are exchanged, is produced that describes, from a **global viewpoint** [...] observable behaviour of all the parties involved. **Each party** can then use the global definition to **build and test solutions that conform to it**. The global specification is in turn **realised by combination of the resulting local systems** [...]"



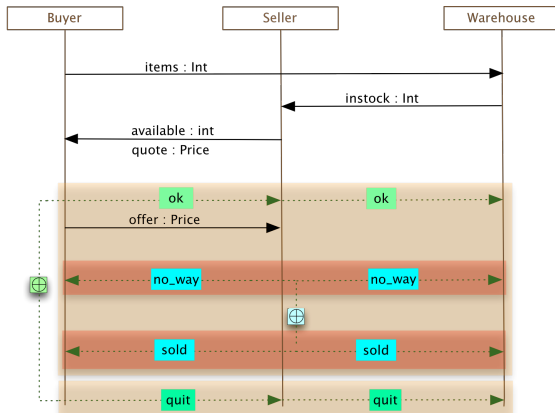
"Top-down" & "bottom-up" [23]

Quoting W3C [21]

"Using the Web Services Choreography specification, a **contract** containing a global definition of the common **ordering conditions and constraints** under which **messages** are exchanged, is produced that describes, from a **global viewpoint** [...] observable behaviour of all the parties involved. **Each party** can then use the global definition to **build and test solutions that conform to it**. The global specification is in turn **realised by combination of the resulting local systems** [...]"

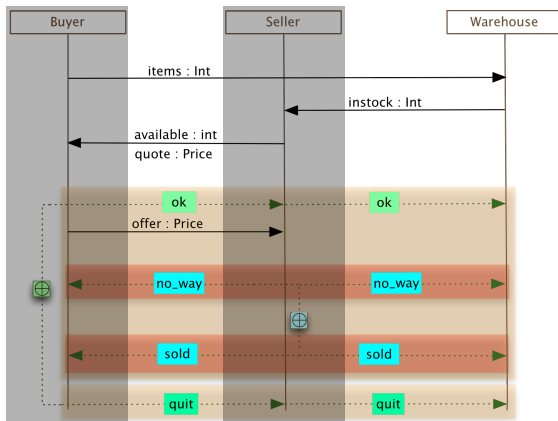


An intuitive account...



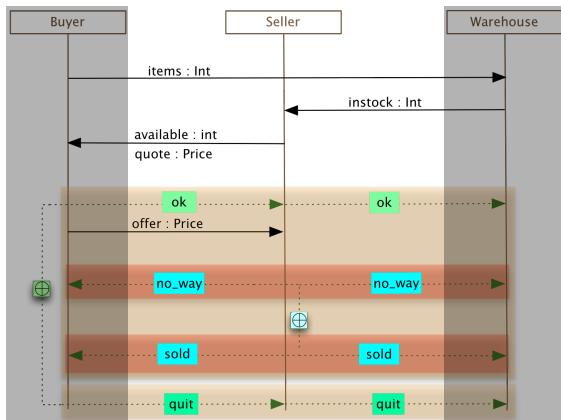
Global viewpoint

An intuitive account...



Projecting on Warehouse

An intuitive account...



Projecting on **seller**

– Design-by-contract of protocols –

A formal global view

global types [20]:

$$\begin{array}{lcl} \mathcal{G} & ::= & A \rightarrow B : m \\ & | & A \rightarrow B : \left(l_j : \mathcal{G}_j \right)_{j \in J} \\ & | & \mu t . \mathcal{G} \\ & | & t \langle \tilde{e} \rangle \\ & | & \text{end} \end{array}$$

A formal global view

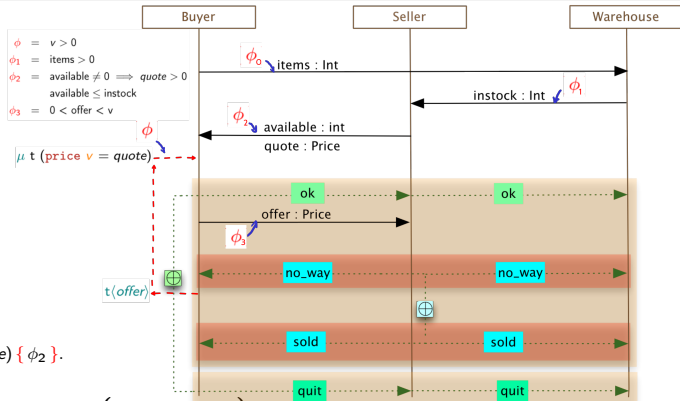
Asserted [2] global types [20]:

$$\begin{array}{lcl} \mathcal{G} & ::= & A \rightarrow B: (\tilde{s} \tilde{x}) \{ \phi \}. \mathcal{G} \\ & | & A \rightarrow B: \left(\{ \phi_j \} l_j: \mathcal{G}_j \right)_{j \in J} \\ & | & \mu t (\tilde{s} \tilde{x} = \tilde{e}) \{ \phi \}. \mathcal{G} \\ & | & t \langle \tilde{e} \rangle \\ & | & \text{end} \end{array}$$

where

- ▶ ϕ and ϕ_j are predicates constraining variables x occurring in the protocol
- ▶ l_j are labels
- ▶ e is an expression denoting values of basic sorts s (eg integer, lists, bool, etc)
- ▶ recursion parameters are initialised with some values

DbC on our buyer-seller-warehouse protocol



$\mathcal{G}_{neg} =$ Buyer $\rightarrow$ Warehouse: (int items) $\{\phi_0\}$.
 Warehouse $\rightarrow$ Seller: (int instock) $\{\phi_1\}$.
 Seller $\rightarrow$ Buyer: (int available, price quote) $\{\phi_2\}$.
 μt (price v = quote) $\{\phi\}$.

Buyer $\rightarrow$ Seller: $\left(\begin{array}{l} \{T\}ok: \text{Buyer} \rightarrow \text{Warehouse}: \left(\begin{array}{l} \{T\}ok: \mathcal{G}_{ok} \end{array} \right), \\ \{T\}quit: \text{Buyer} \rightarrow \text{Warehouse}: \left(\begin{array}{l} \{T\}quit: \text{end} \end{array} \right) \end{array} \right)$

$\mathcal{G}_{ok} =$ Buyer $\rightarrow$ Seller: (price offer) $\{0 < \text{offer} < v\}$.

Seller $\rightarrow$ Buyer: $\left(\begin{array}{l} \{T\}no_way: \text{Seller} \rightarrow \text{Warehouse}: \left(\begin{array}{l} \{T\}no_way: t\langle offer \rangle \end{array} \right), \\ \{T\}sold: \text{Seller} \rightarrow \text{Warehouse}: \left(\begin{array}{l} \{T\}sold: \text{end} \end{array} \right) \end{array} \right)$

Bad specs

Exercise

What's wrong with the following specifications?

$$\mathcal{G}_1 = A \rightarrow B: (\text{int } x) \{ \top \}. B \rightarrow C: (\text{int } y) \{ \top \}. C \rightarrow A: (\text{int } v) \{ v > x \}. \text{end}$$

Exercise

What's wrong with the following specifications?

$\mathcal{G}_1 = A \rightarrow B: (\text{int } x) \{ \top \}. B \rightarrow C: (\text{int } y) \{ \top \}. C \rightarrow A: (\text{int } v) \{ v > x \}. \text{end}$

History Sensitivity In any interaction of $A \rightarrow B: (s \ x) \{ \phi \}. \mathcal{G}$ in G A "knows" all the variables in ϕ

Bad specs

Exercise

What's wrong with the following specifications?

$\mathcal{G}_1 = A \rightarrow B: (\text{int } x) \{ \top \}. B \rightarrow C: (\text{int } y) \{ \top \}. C \rightarrow A: (\text{int } v) \{ v > x \}. \text{end}$

$\mathcal{G}_2 = A \rightarrow B: (\text{int } x) \{ x \geq 6 \}. B \rightarrow A: (\text{int } y) \{ y \leq x \wedge y > 6 \}. \text{end}$

History Sensitivity In any interaction of $A \rightarrow B: (s \ x) \{ \phi \}. \mathcal{G}$ in G A "knows" all the variables in ϕ

Bad specs

Exercise

What's wrong with the following specifications?

$\mathcal{G}_1 = A \rightarrow B: (\text{int } x) \{ \top \}. B \rightarrow C: (\text{int } y) \{ \top \}. C \rightarrow A: (\text{int } v) \{ v > x \}. \text{end}$

$\mathcal{G}_2 = A \rightarrow B: (\text{int } x) \{ x \geq 6 \}. B \rightarrow A: (\text{int } y) \{ y \leq x \wedge y > 6 \}. \text{end}$

History Sensitivity In any interaction of $A \rightarrow B: (s \ x) \{ \phi \}. \mathcal{G}$ in G A "knows" all the variables in ϕ

Temporal Satisfiability Each predicate ϕ in G is satisfiable under the constraints "accumulated" by the predicates of interactions/branchings occurring before A in G

Endpoints & projections

$$\begin{aligned}
 \mathcal{T} ::= & \text{!R}(\text{s } x)\{\phi\}; \mathcal{T} \mid \oplus \text{R} \left(\{\phi_i\}_{i: \mathcal{T}_i} \right)_{i \in I} \\
 & \mid ?\text{S}(\text{s } x)\{\phi\}; \mathcal{T} \mid \&\text{S} \left(\{\phi_i\}_{i: \mathcal{T}_i} \right)_{i \in I} \\
 & \mid \mu \text{t}(\text{s } x = e)\{\phi\}. \mathcal{T} \\
 & \mid \text{t}\langle e \rangle \\
 & \mid \text{end}
 \end{aligned}$$

$$\text{S} \rightarrow \text{R}: (\text{s } x)\{\phi'\}. \mathcal{G}' \downarrow_{\text{A}}^{\phi} = \begin{cases} \text{!R}(\text{s } x)\{\phi'\}; (\mathcal{G}' \downarrow_{\text{A}}^{\phi \wedge \phi'}) & \text{if } \text{A} = \text{S} \\ ?\text{S}(\text{s } x)\{(\phi \wedge \phi') \downarrow_{\text{A}}\}; (\mathcal{G}' \downarrow_{\text{A}}^{\phi \wedge \phi'}) & \text{if } \text{A} = \text{R} \\ \mathcal{G}' \downarrow_{\text{A}}^{\phi \wedge \phi'} & \text{otherwise} \end{cases}$$

$$\text{S} \rightarrow \text{R}: \left(\{\phi_i\}_{i: \mathcal{G}_i} \right)_{i \in I} \downarrow_{\text{A}}^{\phi} = \begin{cases} \oplus \text{R} \left(\{\phi_i\}_{i: \mathcal{G}_i} \downarrow_{\text{A}}^{\phi \wedge \phi_i} \right)_{i \in I} & \text{if } \text{A} = \text{S} \\ \&\text{S} \left(\{(\phi \wedge \phi_i) \downarrow_{\text{A}}\}_{i: \mathcal{G}_i} \downarrow_{\text{A}}^{\phi \wedge \phi_i} \right)_{i \in I} & \text{if } \text{A} = \text{R} \\ \sqcup_{i \in I} \mathcal{G}_i \downarrow_{\text{A}}^{\phi \wedge \phi_i} & \text{otherwise} \end{cases}$$

...

A glimpse of the typing

A glimpse of the typing: initiate

$$\frac{\Gamma \vdash a : \mathcal{G} \quad \mathcal{C} ; \Gamma \vdash P \triangleright \Delta, c : \mathcal{G} \downarrow_A}{\mathcal{C} ; \Gamma \vdash A @ \bar{a}(c). P \triangleright \Delta}$$

A glimpse of the typing: initiate, join

$$\frac{\Gamma \vdash a : \mathcal{G} \quad \mathcal{C} ; \Gamma \vdash P \triangleright \Delta, c : \mathcal{G} \downarrow_A}{\mathcal{C} ; \Gamma \vdash A @ \bar{a}(c).P \triangleright \Delta}$$

$$\frac{\Gamma \vdash a : \mathcal{G} \quad \mathcal{C} ; \Gamma \vdash P \triangleright \Delta, c : \mathcal{G} \downarrow_B}{\mathcal{C} ; \Gamma \vdash B @ a(c).P \triangleright \Delta}$$

A glimpse of the typing: initiate, join, start

$$\frac{\Gamma \vdash a : \mathcal{G} \quad \mathcal{C} ; \Gamma \vdash P \triangleright \Delta, c : \mathcal{G} \downarrow_A}{\mathcal{C} ; \Gamma \vdash A @ \bar{a}(c).P \triangleright \Delta}$$

$$\frac{\Gamma \vdash a : \mathcal{G} \quad \mathcal{C} ; \Gamma \vdash P \triangleright \Delta, c : \mathcal{G} \downarrow_B}{\mathcal{C} ; \Gamma \vdash B @ a(c).P \triangleright \Delta}$$

$$\frac{\Gamma \vdash e : \mathbf{s} \quad \mathcal{C} \supset \phi[e/x] \text{ valid} \quad \mathcal{C} ; \Gamma \vdash P[e/x] \triangleright \Delta, c : \mathcal{T}[e/x]}{\mathcal{C} ; \Gamma \vdash X @ c ! e ; P \triangleright \Delta, c : ! X(\mathbf{s} \ x) \{ \phi \} ; \mathcal{T}}$$

A glimpse of the typing: initiate, join, start

$$\frac{\Gamma \vdash a : \mathcal{G} \quad \mathcal{C} ; \Gamma \vdash P \triangleright \Delta, c : \mathcal{G} \downarrow_A}{\mathcal{C} ; \Gamma \vdash A @ \bar{a}(c).P \triangleright \Delta}$$

$$\frac{\Gamma \vdash a : \mathcal{G} \quad \mathcal{C} ; \Gamma \vdash P \triangleright \Delta, c : \mathcal{G} \downarrow_B}{\mathcal{C} ; \Gamma \vdash B @ a(c).P \triangleright \Delta}$$

$$\frac{\Gamma \vdash e : \mathbf{s} \quad \mathcal{C} \supset \phi[e/x] \text{ valid} \quad \mathcal{C} ; \Gamma \vdash P[e/x] \triangleright \Delta, c : \mathcal{T}[e/x]}{\mathcal{C} ; \Gamma \vdash X @ c ! e ; P \triangleright \Delta, c : ! X(\mathbf{s} \ x) \{ \phi \} ; \mathcal{T}}$$

$$\frac{\mathcal{C} \wedge \phi ; \Gamma, x : \mathbf{s} \vdash P \triangleright \Delta, c : \mathcal{T}}{\mathcal{C} ; \Gamma \vdash X @ c ?(x) ; P \triangleright \Delta, c : ? X(\mathbf{s} \ x) \{ \phi \} ; \mathcal{T}}$$

.....

A glimpse of the typing: initiate, join, start

$$\frac{\Gamma \vdash a : \mathcal{G} \quad \mathcal{C} ; \Gamma \vdash P \triangleright \Delta, c : \mathcal{G} \downarrow_A}{\mathcal{C} ; \Gamma \vdash A @ \bar{a}(c).P \triangleright \Delta}$$

$$\frac{\Gamma \vdash a : \mathcal{G} \quad \mathcal{C} ; \Gamma \vdash P \triangleright \Delta, c : \mathcal{G} \downarrow_B}{\mathcal{C} ; \Gamma \vdash B @ a(c).P \triangleright \Delta}$$

$$\frac{\Gamma \vdash e : s \quad \mathcal{C} \supset \phi[e/x] \text{ valid} \quad \mathcal{C} ; \Gamma \vdash P[e/x] \triangleright \Delta, c : \mathcal{T}[e/x]}{\mathcal{C} ; \Gamma \vdash X @ c ! e ; P \triangleright \Delta, c : ! X(s \ x) \{ \phi \} ; \mathcal{T}}$$

$$\frac{\mathcal{C} \wedge \phi ; \Gamma, x : s \vdash P \triangleright \Delta, c : \mathcal{T}}{\mathcal{C} ; \Gamma \vdash X @ c ?(x) ; P \triangleright \Delta, c : ? X(s \ x) \{ \phi \} ; \mathcal{T}}$$

.....

Thm. Well-typed processes behave according to the protocols they implement

– Semantics of choreographies –

Making sense of global views

Making sense of global views

- ▶ The previous theorem (and the others in [2]) leverage projections & an operational semantics of local types

Making sense of global views

- ▶ The previous theorem (and the others in [2]) leverage projections & an operational semantics of local types
- ▶ This approach has continued for a while (see e.g., [19, 6, 23])

Making sense of global views

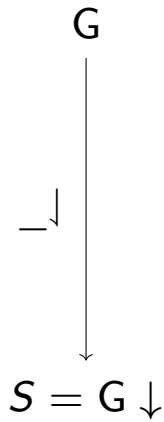
- ▶ The previous theorem (and the others in [2]) leverage projections & an operational semantics of local types
- ▶ This approach has continued for a while (see e.g., [19, 6, 23])
- ▶ Other approaches introduce an operational semantics of global types (see e.g., [5, 22, 20, 9, 10, 3])

Making sense of global views

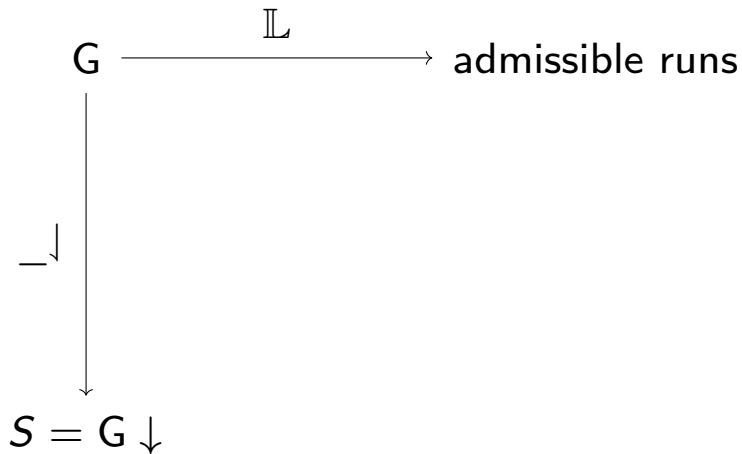
- ▶ The previous theorem (and the others in [2]) leverage projections & an operational semantics of local types
- ▶ This approach has continued for a while (see e.g., [19, 6, 23])
- ▶ Other approaches introduce an operational semantics of global types (see e.g., [5, 22, 20, 9, 10, 3])
- ▶ Recently there has been interest on denotational semantics of global views (see e.g., [25, 16, 7, 14, 12, 8, 13])

G

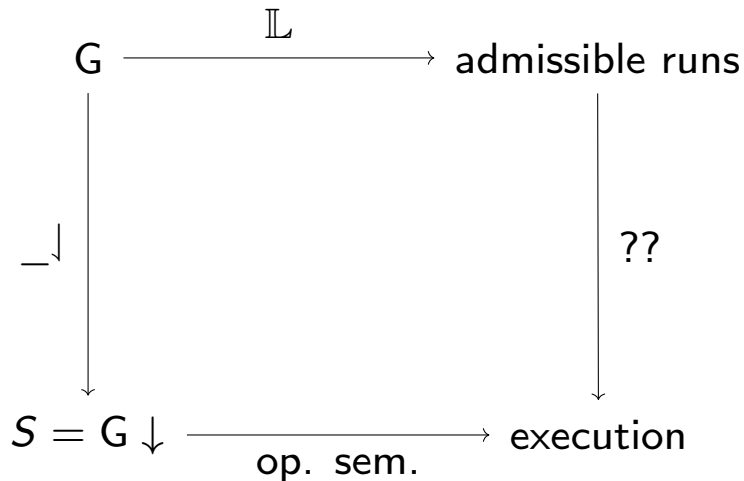
Relating semantics of global views



Relating semantics of global views



Relating semantics of global views



Global views as graphs

G-choreographies [25, 16] are regular expressions (on an alphabet of **interactions**) with parallel composition¹

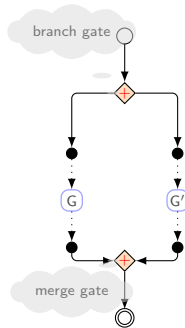
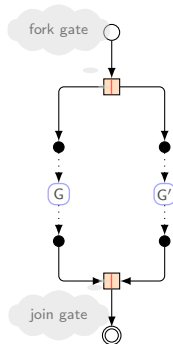
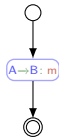
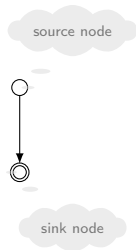
$$G, G' ::= \odot \mid A \rightarrow B : m \mid G \mid G' \mid G ; G' \mid G + G'$$

¹For simplicity, here we omit iteration

Global views as graphs

G-choreographies [25, 16] are regular expressions (on an alphabet of **interactions**) with parallel composition¹

$$G, G' ::= \odot \mid A \rightarrow B : m \mid G \mid G' \mid G ; G' \mid G + G'$$



¹For simplicity, here we omit iteration

A pomsets semantics

A pomset on a set $\mathcal{E}$ of events is (an isomorphism class of) a labelled partially-order $(\mathcal{E}, \lambda, \leq)$, with

- ▶ $\lambda : \mathcal{E} \rightarrow \mathcal{L}$ a labelling function
- ▶ $\leq \subseteq \mathcal{E} \times \mathcal{E}$ reflexive, anti-symmetric, transitive

A pomsets semantics

A pomset on a set $\mathcal{E}$ of events is (an isomorphism class of) a labelled partially-order $(\mathcal{E}, \lambda, \leq)$, with

- ▶ $\lambda : \mathcal{E} \rightarrow \mathcal{L}$ a labelling function
- ▶ $\leq \subseteq \mathcal{E} \times \mathcal{E}$ reflexive, anti-symmetric, transitive

Our basic ingredients:

A pomsets semantics

A pomset on a set $\mathcal{E}$ of events is (an isomorphism class of) a labelled partially-order $(\mathcal{E}, \lambda, \leq)$, with

- ▶ $\lambda : \mathcal{E} \rightarrow \mathcal{L}$ a labelling function
- ▶ $\leq \subseteq \mathcal{E} \times \mathcal{E}$ reflexive, anti-symmetric, transitive

Our basic ingredients:

- ▶ a set $\mathcal{M}$ of messages

A pomsets semantics

A pomset on a set $\mathcal{E}$ of events is (an isomorphism class of) a labelled partially-order $(\mathcal{E}, \lambda, \leq)$, with

- ▶ $\lambda : \mathcal{E} \rightarrow \mathcal{L}$ a labelling function
- ▶ $\leq \subseteq \mathcal{E} \times \mathcal{E}$ reflexive, anti-symmetric, transitive

Our basic ingredients:

- ▶ a set $\mathcal{M}$ of messages
- ▶ a set $\mathcal{P}$ of participants' identities

A pomsets semantics

A pomset on a set $\mathcal{E}$ of events is (an isomorphism class of) a labelled partially-order $(\mathcal{E}, \lambda, \leq)$, with

- ▶ $\lambda : \mathcal{E} \rightarrow \mathcal{L}$ a labelling function
- ▶ $\leq \subseteq \mathcal{E} \times \mathcal{E}$ reflexive, anti-symmetric, transitive

Our basic ingredients:

- ▶ a set $\mathcal{M}$ of messages
- ▶ a set $\mathcal{P}$ of participants' identities
- ▶ a set $\mathcal{C} = \mathcal{P} \times \mathcal{P} \setminus \{(A, A) \mid A \in \mathcal{P}\}$ of channels

A pomsets semantics

A pomset on a set $\mathcal{E}$ of events is (an isomorphism class of) a labelled partially-order $(\mathcal{E}, \lambda, \leq)$, with

- ▶ $\lambda : \mathcal{E} \rightarrow \mathcal{L}$ a labelling function
- ▶ $\leq \subseteq \mathcal{E} \times \mathcal{E}$ reflexive, anti-symmetric, transitive

Our basic ingredients:

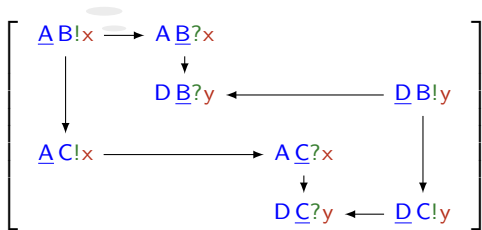
- ▶ a set $\mathcal{M}$ of messages
- ▶ a set $\mathcal{P}$ of participants' identities
- ▶ a set $\mathcal{C} = \mathcal{P} \times \mathcal{P} \setminus \{(A, A) \mid A \in \mathcal{P}\}$ of channels

Our labels : $\mathcal{L} = \mathcal{L}^! \cup \mathcal{L}^?$ where

$$\mathcal{L}^! = \mathcal{C} \times \{!\} \times \mathcal{M} \quad \text{and} \quad \mathcal{L}^? = \mathcal{C} \times \{?\} \times \mathcal{M}$$

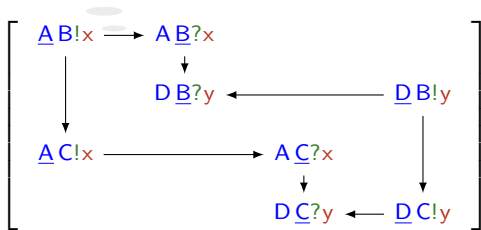
A simple pomset of communication events

meaning $e \leq e'$



A simple pomset of communication events

meaning $e \leq e'$



Exercise (An unfair question!)

Find a g-choreography that "corresponds" to the pomset on the left.

From languages of pomsets

The language $\mathbb{L}[r]$ of $r = [\mathcal{E}, \leq, \lambda]$ is the λ -image

if $w = e_1 e_2 e_3$ *and*
 $\lambda(e_i) = A_i B!m$ *then*
 $\lambda(w) = \underline{A_1} B!m \underline{A_2} B!m \underline{A_3} B!m$

From languages of pomsets to languages of choreographies

The language $\mathbb{L}[r]$ of $r = [\mathcal{E}, \leq, \lambda]$ is the λ -image of the (prefixes of) linearisations of r , namely of words $w \in \mathcal{E}^*$ such that for any $0 \leq i \neq j \leq \text{len}(w)$

No duplication $w[i] \neq w[j]$

Causally compliant if $i \leq j$ then $w[j] \not\leq w[i]$

Downward closed for every $e \in \mathcal{E}$, if there is i s.t. $e < w[i]$ then $e \in w$

From languages of pomsets to languages of choreographies

The language $\mathbb{L}[r]$ of $r = [\mathcal{E}, \leq, \lambda]$ is the λ -image of the (prefixes of) linearisations of r , namely of words $w \in \mathcal{E}^*$ such that for any $0 \leq i \neq j \leq \text{len}(w)$

No duplication $w[i] \neq w[j]$

Causally compliant if $i \leq j$ then $w[j] \not\prec w[i]$

Downward closed for every $e \in \mathcal{E}$, if there is i s.t. $e < w[i]$ then $e \in w$

The language of $G \in \mathcal{G}$ is

$$\mathbb{L}[G] = \begin{cases} \bigcup_{r \in \llbracket G \rrbracket} \mathbb{L}[r], & \text{if } \llbracket G \rrbracket \text{ is defined} \\ \text{undef}, & \text{otherwise} \end{cases}$$

Pomsets semantics of g-choreographies

We define $\llbracket \cdot \rrbracket : G \mapsto \text{set of pomsets}$ as

$$\llbracket \odot \rrbracket = \{\epsilon\}$$

$$\llbracket A \rightarrow B : m \rrbracket = \left\{ \left[\begin{array}{c} \textcolor{blue}{A} \textcolor{blue}{B} ! \textcolor{red}{m} \rightarrow \textcolor{blue}{A} \textcolor{blue}{B} ? \textcolor{red}{m} \end{array} \right] \right\}$$

$$\llbracket G \mid G' \rrbracket = \{ [\text{disjoint union of } r \text{ and } r'] \mid (r, r') \in \llbracket G \rrbracket \times \llbracket G' \rrbracket \}$$

$$\llbracket G ; G' \rrbracket = \begin{cases} \bigcup_{(r, r') \in \llbracket G \rrbracket \times \llbracket G' \rrbracket} \{ \textcolor{teal}{seq}(r, r') \} & \text{if } \textcolor{red}{ws}(G, G') \\ \text{undef} & \text{otherwise} \end{cases}$$

$$\llbracket G + G' \rrbracket = \begin{cases} \llbracket G \rrbracket \cup \llbracket G' \rrbracket & \text{if } \textcolor{red}{wb}(G, G') \\ \text{undef} & \text{otherwise} \end{cases}$$

Pomsets semantics of g-choreographies

We define $\llbracket \cdot \rrbracket : G \mapsto \text{set of pomsets as}$

$$\llbracket \odot \rrbracket = \{\epsilon\}$$

$$\llbracket A \rightarrow B : m \rrbracket = \{ \llbracket A \rrbracket m \rightarrow \llbracket B \rrbracket m \}$$

*Assuming
single-
threadness*

$$\llbracket G \mid G' \rrbracket = \{ [\text{disjoint union of } r \text{ and } r'] \mid (r, r') \in \llbracket G \rrbracket \times \llbracket G' \rrbracket \}$$

$$\llbracket G ; G' \rrbracket = \begin{cases} \bigcup_{(r, r') \in \llbracket G \rrbracket \times \llbracket G' \rrbracket} \{\text{seq}(r, r')\} & \text{if } \text{ws}(G, G') \\ \text{undef} & \text{otherwise} \end{cases}$$

$$\llbracket G + G' \rrbracket = \begin{cases} \llbracket G \rrbracket \cup \llbracket G' \rrbracket & \text{if } \text{wb}(G, G') \\ \text{undef} & \text{otherwise} \end{cases}$$

Some simple examples

Semantics of g-choreographies

$$G_1 = A \rightarrow B: x \mid A \rightarrow B: y$$

$$\llbracket G_1 \rrbracket = \left\{ \left[\begin{array}{c} \underline{A}B!x \\ \downarrow \\ \underline{A}\underline{B}?x \end{array} \quad \begin{array}{c} \underline{A}B!y \\ \downarrow \\ \underline{A}\underline{B}?y \end{array} \right] \right\}$$

$$G_2 = A \rightarrow B: x; B \rightarrow A: y$$

$$\llbracket G_2 \rrbracket = \left\{ \left[\begin{array}{c} \underline{A}B!x \\ \downarrow \\ \underline{A}\underline{B}?x \end{array} \quad \begin{array}{c} \underline{B}A!y \\ \downarrow \\ \underline{B}A?y \end{array} \right] \right\}$$


$$G_3 = A \rightarrow B: x + A \rightarrow B: y$$

$$\llbracket G_3 \rrbracket = \left\{ \left[\begin{array}{c} \underline{A}B!x \\ \downarrow \\ \underline{A}\underline{B}?x \end{array} \right], \left[\begin{array}{c} \underline{A}B!y \\ \downarrow \\ \underline{A}\underline{B}?y \end{array} \right] \right\}$$

Some simple examples

Semantics of g-choreographies

$$G_1 = A \rightarrow B: x \mid A \rightarrow B: y$$

$$\llbracket G_1 \rrbracket = \left\{ \left[\begin{array}{cc} \underline{A}B!x & \underline{A}B!y \\ \downarrow & \downarrow \\ \underline{A}B?x & \underline{A}B?y \end{array} \right] \right\}$$

$$\begin{aligned} \mathbb{L}[G_1] = \{ & \underline{A}B!x \underline{A}B!y \underline{A}B?x \underline{A}B?y, \\ & \underline{A}B!y \underline{A}B!x \underline{A}B?x \underline{A}B?y, \\ & \underline{A}B!x \underline{A}B!y \underline{A}B?y \underline{A}B?x, \\ & \underline{A}B!y \underline{A}B!x \underline{A}B?y \underline{A}B?x, \\ & \underline{A}B!x \underline{A}B?x \underline{A}B!y \underline{A}B?y, \\ & \underline{A}B!y \underline{A}B?y \underline{A}B!x \underline{A}B?x, \\ & + \text{ all the prefixes} \} \end{aligned}$$

$$G_2 = A \rightarrow B: x; B \rightarrow A: y$$

$$\llbracket G_2 \rrbracket = \left\{ \left[\begin{array}{cc} \underline{A}B!x & \underline{B}A!y \\ \downarrow & \downarrow \\ \underline{A}B?x & \underline{B}A?y \end{array} \right] \right\}$$


$$G_3 = A \rightarrow B: x + A \rightarrow B: y$$

$$\llbracket G_3 \rrbracket = \left\{ \left[\begin{array}{c} \underline{A}B!x \\ \downarrow \\ \underline{A}B?x \end{array} \right], \left[\begin{array}{c} \underline{A}B!y \\ \downarrow \\ \underline{A}B?y \end{array} \right] \right\}$$

Some simple examples

Semantics of g-choreographies

$$G_1 = A \rightarrow B: x \mid A \rightarrow B: y$$

$$\llbracket G_1 \rrbracket = \left\{ \left[\begin{array}{cc} \underline{A}B!x & \underline{A}B!y \\ \downarrow & \downarrow \\ \underline{A}B?x & \underline{A}B?y \end{array} \right] \right\}$$

$$\begin{aligned} \mathbb{L}[G_1] = \{ & \underline{A}B!x \underline{A}B!y \underline{A}B?x \underline{A}B?y, \\ & \underline{A}B!y \underline{A}B!x \underline{A}B?x \underline{A}B?y, \\ & \underline{A}B!x \underline{A}B!y \underline{A}B?y \underline{A}B?x, \\ & \underline{A}B!y \underline{A}B!x \underline{A}B?y \underline{A}B?x, \\ & \underline{A}B!x \underline{A}B?x \underline{A}B!y \underline{A}B?y, \\ & \underline{A}B!y \underline{A}B?y \underline{A}B!x \underline{A}B?x, \\ & + \text{ all the prefixes} \} \end{aligned}$$

$$G_2 = A \rightarrow B: x; B \rightarrow A: y$$

$$\llbracket G_2 \rrbracket = \left\{ \left[\begin{array}{cc} \underline{A}B!x & \underline{B}A!y \\ \downarrow & \downarrow \\ \underline{A}B?x & \underline{B}A?y \end{array} \right] \right\}$$

$$\begin{aligned} \mathbb{L}[G_2] = \{ & \underline{A}B!x \underline{A}B?x \underline{B}A!y \underline{B}A?y \\ & \underline{A}B!x \underline{A}B?x \underline{B}A!y \\ & \underline{A}B!x \underline{A}B?x \\ & \underline{A}B!x \\ & \varepsilon \} \end{aligned}$$

$$G_3 = A \rightarrow B: x + A \rightarrow B: y$$

$$\llbracket G_3 \rrbracket = \left\{ \left[\begin{array}{c} \underline{A}B!x \\ \downarrow \\ \underline{A}B?x \end{array} \right], \left[\begin{array}{c} \underline{A}B!y \\ \downarrow \\ \underline{A}B?y \end{array} \right] \right\}$$

Some simple examples

Semantics of g-choreographies

$$G_1 = A \rightarrow B: x \mid A \rightarrow B: y$$

$$\llbracket G_1 \rrbracket = \left\{ \left[\begin{array}{cc} \underline{A}B!x & \underline{A}B!y \\ \downarrow & \downarrow \\ \underline{A}B?x & \underline{A}B?y \end{array} \right] \right\}$$

$$\begin{aligned} \mathbb{L}[G_1] = \{ & \underline{A}B!x \underline{A}B!y \underline{A}B?x \underline{A}B?y, \\ & \underline{A}B!y \underline{A}B!x \underline{A}B?x \underline{A}B?y, \\ & \underline{A}B!x \underline{A}B!y \underline{A}B?y \underline{A}B?x, \\ & \underline{A}B!y \underline{A}B!x \underline{A}B?y \underline{A}B?x, \\ & \underline{A}B!x \underline{A}B?x \underline{A}B!y \underline{A}B?y, \\ & \underline{A}B!y \underline{A}B?y \underline{A}B!x \underline{A}B?x, \\ & + \text{ all the prefixes} \} \end{aligned}$$

$$G_2 = A \rightarrow B: x; B \rightarrow A: y$$

$$\llbracket G_2 \rrbracket = \left\{ \left[\begin{array}{cc} \underline{A}B!x & \underline{B}A!y \\ \downarrow & \downarrow \\ \underline{A}B?x & \underline{B}A?y \end{array} \right] \right\}$$

$$\begin{aligned} \mathbb{L}[G_2] = \{ & \underline{A}B!x \underline{A}B?x \underline{B}A!y \underline{B}A?y \\ & \underline{A}B!x \underline{A}B?x \underline{B}A!y \\ & \underline{A}B!x \underline{A}B?x \\ & \underline{A}B!x \\ & \varepsilon \} \end{aligned}$$

$$G_3 = A \rightarrow B: x + A \rightarrow B: y$$

$$\llbracket G_3 \rrbracket = \left\{ \left[\begin{array}{c} \underline{A}B!x \\ \downarrow \\ \underline{A}B?x \end{array} \right], \left[\begin{array}{c} \underline{A}B!y \\ \downarrow \\ \underline{A}B?y \end{array} \right] \right\}$$

$$\begin{aligned} \mathbb{L}[G_3] = \{ & \underline{A}B!x \underline{A}B?x, \\ & \underline{A}B!y \underline{A}B?y, \\ & \underline{A}B!x, \\ & \underline{A}B!y, \\ & \varepsilon \} \end{aligned}$$

Some simple examples

Semantics of g-choreographies

$$G_1 = A \rightarrow B: x \mid A \rightarrow B: y$$

$$\llbracket G_1 \rrbracket = \left\{ \left[\begin{array}{cc} \underline{A}B!x & \underline{A}B!y \\ \downarrow & \downarrow \\ \underline{A}B?x & \underline{A}B?y \end{array} \right] \right\}$$

$$\begin{aligned} \mathbb{L}[G_1] = \{ & \underline{A}B!x \underline{A}B!y \underline{A}B?x \underline{A}B?y, \\ & \underline{A}B!y \underline{A}B!x \underline{A}B?x \underline{A}B?y, \\ & \underline{A}B!x \underline{A}B!y \underline{A}B?y \underline{A}B?x, \\ & \underline{A}B!y \underline{A}B!x \underline{A}B?y \underline{A}B?x, \\ & \underline{A}B!x \underline{A}B?x \underline{A}B!y \underline{A}B?y, \\ & \underline{A}B!y \underline{A}B?y \underline{A}B!x \underline{A}B?x, \\ & + \text{ all the prefixes} \} \end{aligned}$$

$$G_2 = A \rightarrow B: x; B \rightarrow A: y$$

$$\llbracket G_2 \rrbracket = \left\{ \left[\begin{array}{cc} \underline{A}B!x & \underline{B}A!y \\ \downarrow & \downarrow \\ \underline{A}B?x & \underline{B}A?y \end{array} \right] \right\}$$

$$\begin{aligned} \mathbb{L}[G_2] = \{ & \underline{A}B!x \underline{A}B?x \underline{B}A!y \underline{B}A?y \\ & \underline{A}B!x \underline{A}B?x \underline{B}A!y \\ & \underline{A}B!x \underline{A}B?x \\ & \underline{A}B!x \\ & \varepsilon \} \end{aligned}$$

$$G_3 = A \rightarrow B: x + A \rightarrow B: y$$

$$\llbracket G_3 \rrbracket = \left\{ \left[\begin{array}{c} \underline{A}B!x \\ \downarrow \\ \underline{A}B?x \end{array} \right], \left[\begin{array}{c} \underline{A}B!y \\ \downarrow \\ \underline{A}B?y \end{array} \right] \right\}$$

$$\begin{aligned} \mathbb{L}[G_3] = \{ & \underline{A}B!x \underline{A}B?x, \\ & \underline{A}B!y \underline{A}B?y, \\ & \underline{A}B!x, \\ & \underline{A}B!y, \\ & \varepsilon \} \end{aligned}$$

Exercise

Now you can verify your answer to the exercise on slide 24.

Some bureaucracy before moving on

$X \uplus Y \triangleq X \times \mathbf{1} \cup Y \times \mathbf{2}$ where

- ▶ $\mathbf{n} \triangleq \{n\}$ for any natural number n
- ▶ $f \otimes \mathbf{n} \triangleq \{(x, n) \mapsto f(x) \mid x \in X\}$ for any f on X
- ▶ $R \otimes \mathbf{n} \triangleq \{((x, n), (y, n)) \mid (x, y) \in R\}$ for any relation $R \subseteq X \times Y$

Given a pomset $r = [\mathcal{E}, \leq, \lambda]$, the pomset of A in r is

$$r|_A = [\mathcal{E}_{r,A}, \leq \cap (\mathcal{E}_{r,A} \times \mathcal{E}_{r,A}), \lambda|_{\mathcal{E}_{r,A}}]$$

where $\mathcal{E}_{r,A} = \{e \in \mathcal{E} \mid \lambda(e) \text{ is performed by } A\}$ is the set of events labelled with actions performed by A .

Well-sequencedness, formally

Let $r = [\mathcal{E}, \leq, \lambda]$ and $r' = [\mathcal{E}', \leq', \lambda']$ and define

$$\begin{aligned}\text{seq}(r, r') &= [\mathcal{E} \uplus \mathcal{E}', \leq_{\text{seq}}, (\lambda \otimes \mathbf{1}) \cup (\lambda' \otimes \mathbf{2})] \\ \leq_{\text{seq}} &= \left((\leq \otimes \mathbf{1}) \cup (\leq' \otimes \mathbf{2}) \cup \bigcup_{A \in \mathcal{P}} ((\mathcal{E}_{r, A} \times \mathbf{1}) \times (\mathcal{E}_{r', A} \times \mathbf{2})) \right)^{\star}\end{aligned}$$

Well-sequencedness, formally

Let $r = [\mathcal{E}, \leq, \lambda]$ and $r' = [\mathcal{E}', \leq', \lambda']$ and define

$$\begin{aligned} \text{seq}(r, r') &= [\mathcal{E} \uplus \mathcal{E}', \leq_{\text{seq}}, (\lambda \otimes \mathbf{1}) \cup (\lambda' \otimes \mathbf{2})] \\ \leq_{\text{seq}} &= \left((\leq \otimes \mathbf{1}) \cup (\leq' \otimes \mathbf{2}) \cup \bigcup_{A \in \mathcal{P}} ((\mathcal{E}_{r,A} \times \mathbf{1}) \times (\mathcal{E}_{r',A} \times \mathbf{2})) \right)^* \end{aligned}$$

Two g-choreographies $G, G' \in \mathcal{G}$ are well-sequenced if

- ▶ $\llbracket G \rrbracket \neq \text{undef}$, $\llbracket G' \rrbracket \neq \text{undef}$, and
- ▶ $ws(r, r')$ for all $r \in \llbracket G \rrbracket$, $r' \in \llbracket G' \rrbracket$

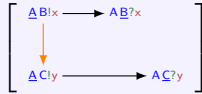
where $ws(r, r')$ if

- ▶ if there are $e \in \mathcal{E}$ and $e' \in \mathcal{E}'$ s.t. $\lambda(e) = A \underline{B} ? m$ and $\lambda'(e') = \underline{A} B ! m$ then $e \leq_{\text{seq}} e'$
- ▶ and $\leq_{\text{seq}(r, r')} \supseteq (\{e \in \mathcal{E} \mid \lambda(e) \in \mathcal{L}^!\} \times \mathbf{1}) \times (\{e \in \mathcal{E}' \mid \lambda'(e) \in \mathcal{L}^?\} \times \mathbf{2})$

Well-sequencedness at work

$A \rightarrow B : x; A \rightarrow C : y$

The "right-input" follows the "left-output"

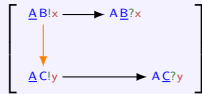


No "right-output"

Well-sequencedness at work

$A \rightarrow B : x; A \rightarrow C : y$

The "right-input" follows the "left-output"



No "right-output"

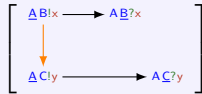
$r = \left[\text{subj } A \quad \text{subj } B \quad \text{subj } C \quad \text{subj } D \right]$

$r' = \left[\text{subj } A \quad \text{subj } B \quad \text{subj } C \right]$

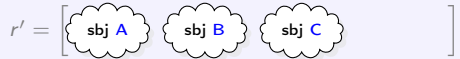
Well-sequencedness at work

$A \rightarrow B : x; A \rightarrow C : y$

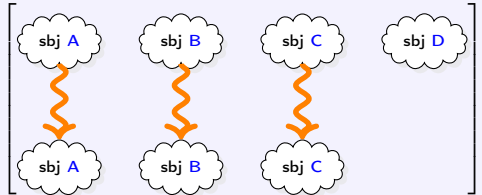
The "right-input" follows the "left-output"



No "right-output"



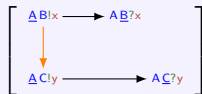
$ws(r, r')$



Well-sequencedness at work

$A \rightarrow B: x; A \rightarrow C: y$

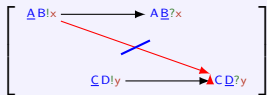
The "right-input" follows the "left-output"



No "right-output"

$A \rightarrow B: x; C \rightarrow D: y$

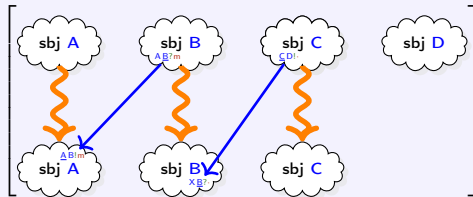
The "right-output" doesn't interfere with "left-inputs"



the "right-input" is not a successor of the "left-output"



$ws(r, r')$



Well-branchedness: an intuition

In a choice $G_1 + G_2$

Well-branchedness: an intuition

In a choice $G_1 + G_2$

- ▶ there should be **at most one active** participant

non-standard

Well-branchedness: an intuition

In a choice $G_1 + G_2$

► there should be **at most one active** participant

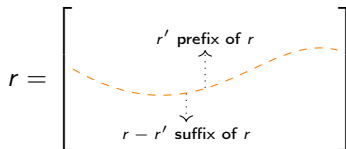
non-standard

► any non-active participant should be **passive**

standard

Problem: Participants do not necessarily "enter" a choice "immediately"

An idea: find a "common part" of the branches for which participants behave uniformly in G_1 and G_2



Some examples

Consider

► $A \rightarrow B: \text{int} + A \rightarrow B: \text{str}$

► $A \rightarrow B: \text{int} + \odot$

► $A \rightarrow B: \text{int} + A \rightarrow C: \text{str}$

► $\left(\begin{array}{l} A \rightarrow C: \text{int}; A \rightarrow B: \text{bool} \\ + \\ A \rightarrow C: \text{str}; A \rightarrow C: \text{bool}; A \rightarrow B: \text{bool} \end{array} \right)$

► $G_{\text{😬}} = G_1 + G_2$

$G_1 = A \rightarrow C: h; A \rightarrow B: l + A \rightarrow C: y; A \rightarrow B: l; A \rightarrow C: z_1$

where $G_2 = A \rightarrow B: r; A \rightarrow C: h + G_3; A \rightarrow B: z_1 + G_3; A \rightarrow B: z_3$

$G_3 = A \rightarrow B: r; A \rightarrow C: y$

Some examples

Consider

► $A \rightarrow B: \text{int} + A \rightarrow B: \text{str}$

► $A \rightarrow B: \text{int} + \odot$

► $A \rightarrow B: \text{int} + A \rightarrow C: \text{str}$

►
$$\left(\begin{array}{l} A \rightarrow C: \text{int}; A \rightarrow B: \text{bool} \\ + \\ A \rightarrow C: \text{str}; A \rightarrow C: \text{bool}; A \rightarrow B: \text{bool} \end{array} \right)$$

► $G_{\text{😬}} = G_1 + G_2$

$$G_1 = A \rightarrow C: h; A \rightarrow B: l + A \rightarrow C: y; A \rightarrow B: l; A \rightarrow C: z_1$$

where $G_2 = A \rightarrow B: r; A \rightarrow C: h + G_3; A \rightarrow B: z_1 + G_3; A \rightarrow B: z_3$

$$G_3 = A \rightarrow B: r; A \rightarrow C: y$$



Some examples

Consider

► $A \rightarrow B: \text{int} + A \rightarrow B: \text{str}$

► $A \rightarrow B: \text{int} + \odot$

► $A \rightarrow B: \text{int} + A \rightarrow C: \text{str}$

►
$$\left(\begin{array}{l} A \rightarrow C: \text{int}; A \rightarrow B: \text{bool} \\ + \\ A \rightarrow C: \text{str}; A \rightarrow C: \text{bool}; A \rightarrow B: \text{bool} \end{array} \right)$$

► $G_{\text{😓}} = G_1 + G_2$

$$G_1 = A \rightarrow C: h; A \rightarrow B: l + A \rightarrow C: y; A \rightarrow B: l; A \rightarrow C: z_1$$

where $G_2 = A \rightarrow B: r; A \rightarrow C: h + G_3; A \rightarrow B: z_1 + G_3; A \rightarrow B: z_3$

$$G_3 = A \rightarrow B: r; A \rightarrow C: y$$



Some examples

Consider

► $A \rightarrow B: \text{int} + A \rightarrow B: \text{str}$

► $A \rightarrow B: \text{int} + \odot$

► $A \rightarrow B: \text{int} + A \rightarrow C: \text{str}$

► $\left(\begin{array}{l} A \rightarrow C: \text{int}; A \rightarrow B: \text{bool} \\ + \\ A \rightarrow C: \text{str}; A \rightarrow C: \text{bool}; A \rightarrow B: \text{bool} \end{array} \right)$

► $G_{\text{😊}} = G_1 + G_2$

$G_1 = A \rightarrow C: h; A \rightarrow B: l + A \rightarrow C: y; A \rightarrow B: l; A \rightarrow C: z_1$

where $G_2 = A \rightarrow B: r; A \rightarrow C: h + G_3; A \rightarrow B: z_1 + G_3; A \rightarrow B: z_3$

$G_3 = A \rightarrow B: r; A \rightarrow C: y$



Some examples

Consider

► $A \rightarrow B: \text{int} + A \rightarrow B: \text{str}$

► $A \rightarrow B: \text{int} + \odot$

► $A \rightarrow B: \text{int} + A \rightarrow C: \text{str}$

►
$$\left(\begin{array}{l} A \rightarrow C: \text{int}; A \rightarrow B: \text{bool} \\ + \\ A \rightarrow C: \text{str}; A \rightarrow C: \text{bool}; A \rightarrow B: \text{bool} \end{array} \right)$$

► $G_{\text{😬}} = G_1 + G_2$

where $G_1 = A \rightarrow C: h; A \rightarrow B: l + A \rightarrow C: y; A \rightarrow B: l; A \rightarrow C: z_1$
 $G_2 = A \rightarrow B: r; A \rightarrow C: h + G_3; A \rightarrow B: z_1 + G_3; A \rightarrow B: z_3$
 $G_3 = A \rightarrow B: r; A \rightarrow C: y$



later

Prefix maps & divergence points

The pair of functions (ϕ, ψ) is an A-prefix map of G_1 and G_2 if

$$\begin{array}{ccc} \text{Part}(\llbracket G_1 \rrbracket|_{\mathbf{A}}) & \xrightarrow{\psi} & \mathcal{P}(\text{Pref}(\llbracket G_1 \rrbracket)) \cap \mathcal{P}(\text{Pref}(\llbracket G_2 \rrbracket)) \\ \text{bijection } \phi \downarrow & & \\ & & \text{Part}(\llbracket G_2 \rrbracket|_{\mathbf{A}}) \end{array}$$

Prefix maps & divergence points

The pair of functions (ϕ, ψ) is an A-prefix map of G_1 and G_2 if

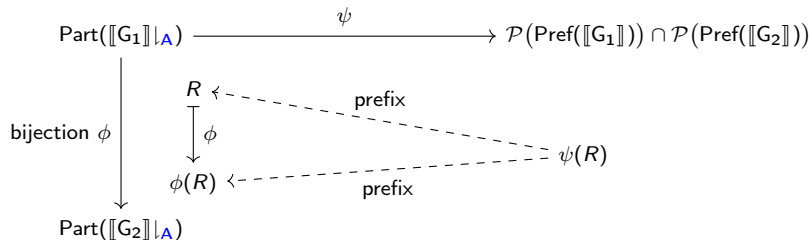
$$\begin{array}{ccc} \text{Part}(\llbracket G_1 \rrbracket|_{\mathbf{A}}) & \xrightarrow{\psi} & \mathcal{P}(\text{Pref}(\llbracket G_1 \rrbracket)) \cap \mathcal{P}(\text{Pref}(\llbracket G_2 \rrbracket)) \\ \downarrow \text{bijection } \phi & & \\ \text{Part}(\llbracket G_2 \rrbracket|_{\mathbf{A}}) & & \end{array}$$

R
 $\downarrow \phi$
 $\phi(R)$

$\psi(R)$

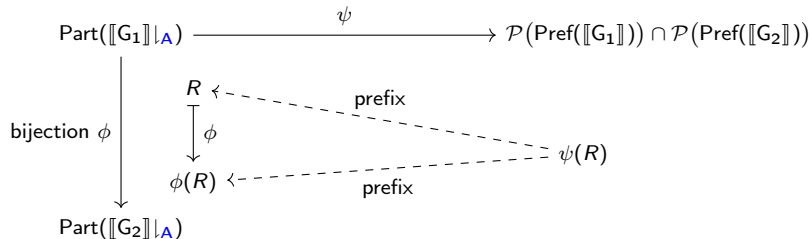
Prefix maps & divergence points

The pair of functions (ϕ, ψ) is an A-prefix map of G_1 and G_2 if



Prefix maps & divergence points

The pair of functions (ϕ, ψ) is an A-prefix map of G_1 and G_2 if



The divergence point of G_1 and G_2 wrt the A-prefix map (ϕ, ψ) is

$$\text{div}_{\phi, \psi}(G_1, G_2) = (L_1, L_2) \quad \text{where} \quad \begin{cases} L_1 = \bigcup_{R \in \text{dom } \phi} \bigcup_{r \in R} \lambda_r(\min(r - \psi(R))) \\ L_2 = \bigcup_{R \in \text{cod } \phi} \bigcup_{r \in R} \lambda_r(\min(r - \psi(\phi^{-1}(R)))) \end{cases}$$

Active & passive components

A is active in $G_1 + G_2$ if there exists an A-prefix map (ϕ, ψ) of G_1 and G_2 with divergence point $\text{div}_{\phi, \psi}(G_1, G_2) = (L_1, L_2)$ s.t.

$$L_1 \cup L_2 \subseteq \mathcal{L}^! \quad L_1 \neq \emptyset \text{ and } L_2 \neq \emptyset \quad L_1 \cap L_2 = \emptyset$$

Active & passive components

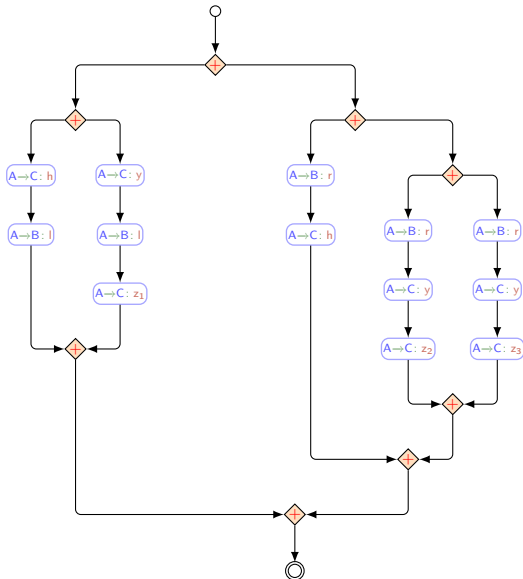
A is active in $G_1 + G_2$ if there exists an A -prefix map (ϕ, ψ) of G_1 and G_2 with divergence point $\text{div}_{\phi, \psi}(G_1, G_2) = (L_1, L_2)$ s.t.

$$L_1 \cup L_2 \subseteq \mathcal{L}^! \quad L_1 \neq \emptyset \text{ and } L_2 \neq \emptyset \quad L_1 \cap L_2 = \emptyset$$

A is passive in $G_1 + G_2$ if there exists an A -prefix map (ϕ, ψ) of G_1 and G_2 with divergence point $\text{div}_{\phi, \psi}(G_1, G_2) = (L_1, L_2)$ s.t.

$$\begin{aligned} L_1 \cup L_2 &\subseteq \mathcal{L}^? & L_1 = \emptyset &\iff L_2 = \emptyset \\ \lambda_r(\mathcal{E}_{r-\psi(R)}) \cap L_2 &= \emptyset \text{ for all } R \in \text{dom } \phi \text{ and } r \in R \\ \lambda_r(\mathcal{E}_{r-\psi(\phi^{-1}(R))}) \cap L_1 &= \emptyset \text{ for all } R \in \text{cod } \phi \text{ and } r \in R \end{aligned}$$

: a difficult choice



$G_{\text{smiley}} = G_1 + G_2$ where

$G_1 = A \rightarrow C: h; A \rightarrow B: l + A \rightarrow C: y; A \rightarrow B: l; A \rightarrow C: z_1$

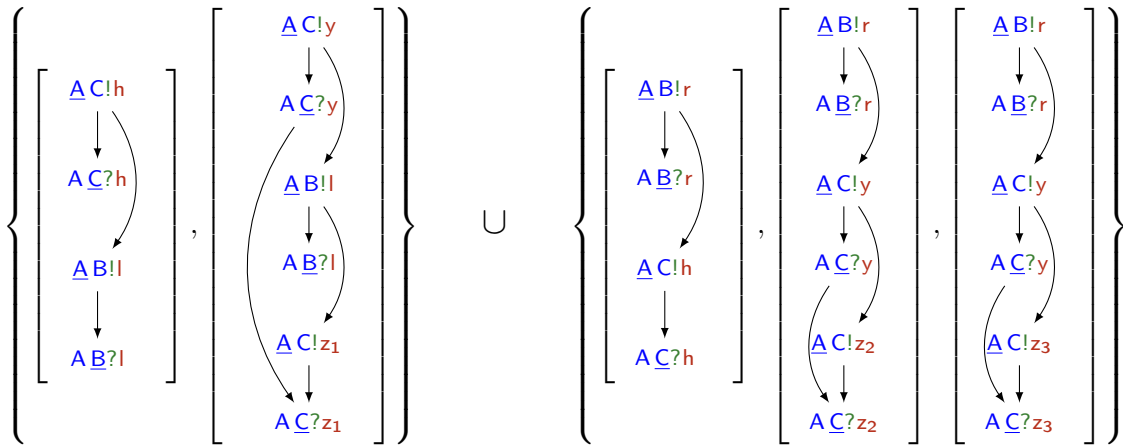
$G_2 = A \rightarrow B: r; A \rightarrow C: h + G_3; A \rightarrow B: z_1 + G_3; A \rightarrow B: z_3$

$G_3 = A \rightarrow B: r; A \rightarrow C: y$

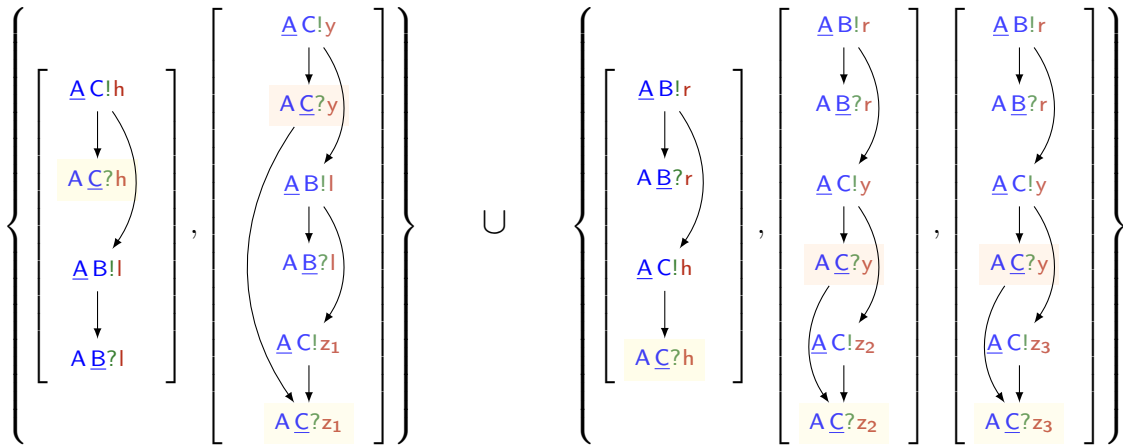
A chooses ... and

- ▶ Whatever B gets, he won't know if A and C exchanged or not **h**
- ▶ Whatever C gets, he won't know if A and B exchanged **l** or **r**

The pomsets of G 😊



The pomsets of G...from C's point of view



Thm If $G \in \mathcal{G}$ with $\llbracket G \rrbracket \neq \perp$ and $S = (\Delta(G \downarrow_A))_{A \in \mathcal{P}}$ then

- ▶ S is deadlock-free
- ▶ $\mathbb{L}[S] \subseteq \mathbb{L}[G]$

The semantics are not equivalent

Projected communicating system can have less behaviours than their protocol due to the FIFO policy on buffers.

Example

Take the g-choreography

$$G = A \rightarrow B : x \mid A \rightarrow B : y$$

we have

$$\underline{A} B ! x \underline{A} B ! y \underline{A} B ? y \underline{A} B ? x \in \mathbb{L}[G] \setminus \mathbb{L}[(\Delta(G \downarrow_A))_{A \in \mathcal{P}}]$$

Thank you!

References I

- [1] Gul Agha. *Actors: A Model of Concurrent Computation in Distributed Systems*. MIT Press, Cambridge, MA, USA, 1986.
- [2] Laura Bocchi, Kohei Honda, Emilio Tuosto, and Nobuko Yoshida. A theory of design-by-contract for distributed multiparty interactions.
In Paul Gastin and François Laroussinie, editors, *CONCUR'10*, volume 6269 of *LNCS*, pages 162–176. Springer, 2010.
- [3] Laura Bocchi, Hernán C. Melgratti, and Emilio Tuosto. On resolving non-determinism in choreographies.
Logical Methods in Computer Science, 16(3), September 2020.
- [4] Daniel Brand and Pitro Zafiropulo. On Communicating Finite-State Machines.
JACM, 30(2):323–342, 1983.
- [5] Mario Bravetti and Gianluigi Zavattaro. Towards a unifying theory for choreography conformance and contract compliance.
In *Proceedings of the 6th International Conference on Software Composition, SC'07*, pages 34–50. Springer-Verlag, 2007.
- [6] Marco Carbone and Fabrizio Montesi. Deadlock-freedom-by-design: multiparty asynchronous global programming.
In *POPL*, pages 263–274, 2013.
- [7] Ilaria Castellani, Mariangiola Dezani-Ciancaglini, and Paola Giannini. Event structure semantics for multiparty sessions.
In *Models, Languages, and Tools for Concurrent and Distributed Programming - Essays Dedicated to Rocco De Nicola on the Occasion of His 65th Birthday*, volume 11665 of *Lecture Notes in Computer Science*, pages 340–363. Springer, 2019.
- [8] Ilaria Castellani, Mariangiola Dezani-Ciancaglini, and Paola Giannini. Event structure semantics for multiparty sessions.
Journal of Logic and Algebraic Methods in Programming, 131:100844, 2023.
- [9] Mario Coppo, Mariangiola Dezani-Ciancaglini, Nobuko Yoshida, and Luca Padovani. Global progress for dynamically interleaved multiparty sessions.
Mathematical Structures in Computer Science, 26(2):238–302, 2016.

References II

- [10] Mila Dalla Preda, Maurizio Gabbriellini, Saverio Giallorenzo, Ivan Lanese, and Mauro Jacopo. Dynamic choreographies - safe runtime updates of distributed applications.
Logical Methods in Computer Science, 13(2), 2017.
- [11] Pierre-Malo Deniérou and Nobuko Yoshida. Multiparty session types meet communicating automata.
In *ESOP 2012*, pages 194–213, 2012.
- [12] Luc Edixhoven and Sung-Shik Jongmans. Realisability of branching pomsets.
In Silvia Lizeth Tapia Tarifa and José Proença, editors, *Formal Aspects of Component Software - 18th International Conference, FACS 2022, Virtual Event, November 10-11, 2022, Proceedings*, volume 13712 of *Lecture Notes in Computer Science*, pages 185–204. Springer, 2022.
- [13] Luc Edixhoven, Sung-Shik Jongmans, José Proença, and Ilaria Castellani. Branching pomsets: Design, expressiveness and applications to choreographies.
J. Log. Algebraic Methods Program., 136:100919, 2024.
- [14] Luc Edixhoven, Sung-Shik Jongmans, José Proença, and Guillermina Cledou. Branching pomsets for choreographies.
In Clément Aubert, Cinzia Di Giusto, Larisa Safina, and Alceste Scalas, editors, *Proceedings 15th Interaction and Concurrency Experience, ICE 2022, Lucca, Italy, 17th June 2022, EPTCS*, pages 37–52, 2022.
- [15] Roberto Guanciale and Emilio Tuosto. An abstract semantics of the global view of choreographies.
In *Proceedings 9th Interaction and Concurrency Experience, ICE 2016, Heraklion, Greece, 8-9 June 2016.*, pages 67–82, 2016.
- [16] Roberto Guanciale and Emilio Tuosto. Realisability of pomsets.
Journal of Logic and Algebraic Methods in Programming, 108:69–89, 2019.

References III

- [17] Carl Hewitt, Peter Boehler Bishop, and Richard Steiger. A Universal Modular ACTOR Formalism for Artificial Intelligence. In Nils J. Nilsson, editor, *Proceedings of the 3rd International Joint Conference on Artificial Intelligence. Stanford, CA, USA, August 20-23, 1973*, pages 235–245. William Kaufmann, 1973.
- [18] Kohei Honda, Vasco Thudichum Vasconcelos, and Makoto Kubo. Language primitives and type discipline for structured communication-based programming. In *ESOP*, 1998.
- [19] Kohei Honda, Nobuko Yoshida, and Marco Carbone. Multiparty asynchronous session types. In *POPL*, POPL '08, page 273–284, New York, NY, USA, 2008. Association for Computing Machinery.
- [20] Kohei Honda, Nobuko Yoshida, and Marco Carbone. Multiparty asynchronous session types. *JACM*, 63(1):9:1–9:67, 2016. Extended version of [19].
- [21] Nickolas Kavantzaz, Davide Burdett, Gregory Ritzinger, Tony Fletcher, and Yves Lafon. Web services choreography description language version 1.0. <http://www.w3.org/TR/2004/WD-ws-cdl-10-20041217>. Working Draft 17 December 2004.
- [22] Ivan Lanese, Claudio Guidi, Fabrizio Montesi, and Gianluigi Zavattaro. Bridging the gap between interaction- and process-oriented choreographies. In *Software Engineering and Formal Methods, SEFM 2008*, pages 323–332, 2008.
- [23] Julien Lange, Emilio Tuosto, and Nobuko Yoshida. From Communicating Machines to Graphical Choreographies. In *POPL15*, pages 221–232, 2015.
- [24] Davide Sangiorgi and David Walker. *The π -Calculus: a Theory of Mobile Processes*. Cambridge University Press, 2002.

References IV

- [25] Emilio Tuosto and Roberto Guanciale. Semantics of global view of choreographies. *Journal of Logic and Algebraic Methods in Programming*, 95:17–40, 2018. Revised and extended version of [15]. available at <http://www.cs.le.ac.uk/people/et52/jlamp-with-proofs.pdf>.